

New York University — Tandon School of Engineering
Department of Technology Management and Innovation
IE-GY 9113B — Systems Integration: From ERP to Agentic AI
Study Note — Week 4: RPA — The Tactical Integration Layer
Module 2: The Automation Ladder | Friday, September 25, 2026

Readings for this week	
Primary case	Thermo Fisher Scientific — RPA and machine learning in customer care and order entry (Project Northstar), Thor Olavsrud, <i>CIO</i> , 12 Aug 2022. Open access: https://www.cio.com/article/404535/thermo-fisher-transforms-its-customer-experience.html
Supporting	Bailey Hydraulics / NetSuite — a working Order-to-Cash baseline at a mid-market manufacturer (the second course archetype). Oracle NetSuite, open access: https://www.netsuite.com/portal/resource/articles/ecommerce/bailey-hydraulics-pumps-up-ecommerce-business-with-netsuite.shtml
Go deeper (optional)	Tella et al., <i>The New Automation Mindset</i> , Ch. 1–2 (the automation mindset and the process mindset). Reference, not required reading; dip in where you want the original voice — the note teaches the idea directly. Deloitte, <i>Automation with intelligence</i> (2022) — the 7th Global Intelligent Automation survey (479 executives, 35 countries), for the state of practice and the business case: https://www.deloitte.com/us/en/insights/topics/talent/intelligent-automation-2022-survey-results.html MRP / S&OP context — Orlicky (1975) on Materials Requirements Planning; Ling & Goddard (1988) on Sales & Operations Planning. Origin references only.
Before next session	No new setup this week; your Lab 1 map is your worksheet for the RPA-candidate scan. (Make accounts come before Lab 2 in Week 6.)

You Have Built the Floor and Named the Gap — Now You Start to Climb

This course moves through four stages — integrate, automate, make intelligent, make agentic — and everything you build sits on the first. Module 1 gave you the floor: Week 0 set the ERP floor, Week 1 defined the integration backbone as a single source of truth, and Week 2 named the leverage gap — the distance between a backbone that works and a backbone that pays — and prescribed its cure, obliterate then automate. Module 2 is where that second verb finally arrives. This week you take the first, most tactical step up the ladder: automation. And the first rung of automation is Robotic Process Automation, or RPA.

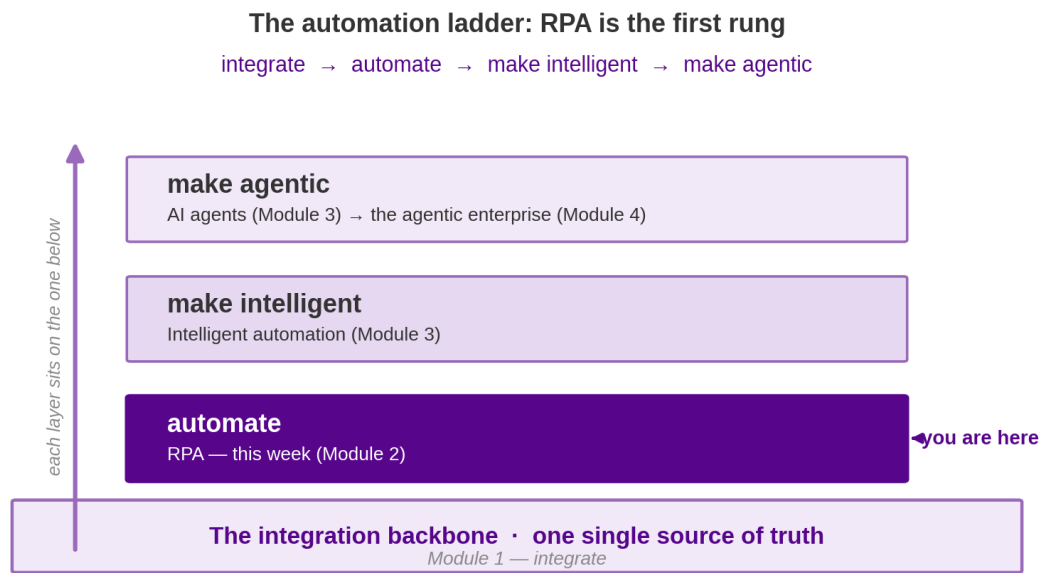


Figure 1. Where RPA sits: the first rung of automation, resting on the Module 1 backbone.

Keep the three threads in your ear as a recurring refrain, because Module 2 leads with one of them. The Customer thread is taking an order and serving the person as one coherent promise; the Operations and Supply Chain thread is planning, inventory, and synchronization, so a company can deliver what it promised; and the Cash Management thread is how fast and how reliably the backbone turns activity into cash — the thread that matters because cash flow is the most common way businesses fail. RPA finds its first traction on the Operations and Supply Chain thread, because that is where the repetitive, rules-based work lives. But every automation decision you make this module reaches the other two: cycle time is cash conversion speed, and order accuracy is customer experience. One robot, three threads.

And keep the four archetypes in view, because you will read this week against all of them: a consumer-packaged-goods manufacturer running sales-and-operations planning on SAP; a mid-market manufacturer running order-to-cash on NetSuite; a Series-B startup running order-to-cash across Stripe, Shopify, and QuickBooks; and a fifty-person services firm living in HubSpot and Notion. This week's supporting case, Bailey Hydraulics, is the second archetype made concrete — a mid-market manufacturer running order-to-cash on NetSuite — so watch how the lesson lands there first.

One orientation before the concepts, so you place this week correctly in the capstone. Identifying an RPA candidate is still diagnostic work — you are scanning a process you already mapped for the steps worth automating, not yet building anything. The build comes in Lab 2. So this week belongs to the same Diagnose phase as Weeks 1 through 3: you are still answering “what is the problem, and where does automation add value?”, now with a sharper tool for spotting where.

RPA is the first rung, not the whole climb. Module 1 built the backbone and named the leverage gap; Module 2 begins to close it by automating. RPA is the first, most tactical layer of automation, and it finds its first traction on Operations and Supply Chain — while every automation decision still moves Cash and Customer. This week you diagnose where a bot belongs; you build one in Lab 2.

The Week's Question: Which Repeatable Steps Deserve a Robot — and Which Will Break One?

Here is the question the whole session turns on. Once a backbone is working, some of the work running through it is genuinely repetitive and rule-bound — the same keystrokes, the same lookups, the same copy-from-one-screen-paste-into-another, thousands of times a month. A robot can do that work faster, cheaper, and without fatigue. But not every repetitive-looking step is a safe candidate, and a bot pointed at the wrong step does real damage: it locks a broken process in place, or it shatters the first time a screen it depends on moves. So the question is two-sided. Which steps are worth handing to a bot — and, just as important, which ones will break one, or should never have existed in the shape they are in? Answering that well is the entire skill this week teaches, and it is exactly the judgment your Board Memo §4 will ask you to demonstrate on your own company.

A Robot That Works the Screens, Not the Plumbing

Start with what RPA actually is, because the word “robot” misleads almost everyone. There is no physical machine and, in the purest form, no deep engineering. Robotic Process Automation is software that operates a company's existing applications the same way a person does — through the user interface, clicking the buttons, reading the fields, copying a value out of one system and typing it into another, following a fixed script of steps. It is, quite literally, a robot that works the screens. It logs into the same order-entry system a clerk uses, reads the same fields, and moves the same data to the same places — just without a human doing the clicking. In practice a bot like this is built on a dedicated RPA platform — UiPath is the market archetype, and the one you will see demonstrated in the session — though what matters for this course is the pattern, not the vendor: you build your own automation on the default no-code tool, Make, in Lab 2.

That surface-level nature is the source of both RPA's appeal and its fragility, so hold both halves at once. The appeal: because a bot reaches across systems at the surface, it can stitch together applications that were never designed to talk to each other, without anyone building a proper integration underneath. Two systems with no shared connection — a legacy order tool and a modern finance package, say — can be bridged by a bot that reads one screen and types into the other, and it can be stood up in days or weeks rather than the months a deep integration would take. This is why RPA spread so fast: it automates the human's job of moving data between systems, which is precisely the symptom of a missing foundation that Week 1 taught you to listen for.

The fragility is the same fact seen from the other side. Because the bot depends on the screen looking exactly as it did when the bot was built, it breaks when the screen changes. A vendor ships an update that moves a field, renames a button, or adds a pop-up, and the bot — which cannot improvise the way a person glancing at the new layout would — either stops or, worse, keeps going and does the wrong thing confidently. Remember that failure mode; it returns in Week 5 as the reason RPA programmes stall at scale, and it is the reason a bot is a tactical patch, not a structural fix. RPA does not repair the missing integration underneath; it papers over it, quickly and cheaply, at the cost of a brittle seam you now have to maintain.

RPA operates the user interface, not the underlying systems. A software bot mimics a person clicking through existing applications, moving data between screens by a fixed script. That surface approach is its strength — it bridges systems that were never integrated, fast and cheap — and its weakness: it breaks when a screen it depends on changes. RPA papers over a missing integration; it does not build one.

Three Ways to Take a Human Out of the Loop

RPA is easiest to understand by what it is not, so set it beside the two things it is constantly confused with: traditional back-end automation on one side, and AI on the other. They differ along one axis that matters more than any other — where in the system each one acts, and what kind of input it can handle.

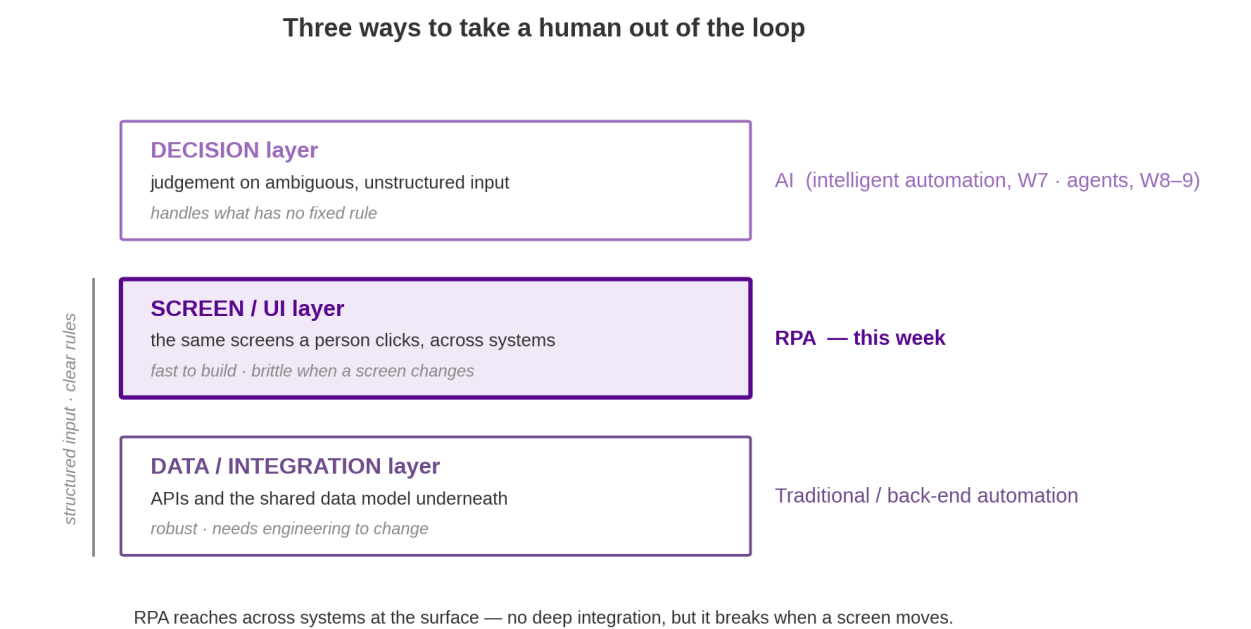


Figure 2. RPA works at the surface of a system; traditional automation works in the plumbing; AI works on judgement.

Traditional automation works in the plumbing. It connects systems through their proper interfaces — an API, a database write, a native integration built by engineers — so that a transaction flows from one system to another underneath the user interface, never touching a

screen. When Bailey Hydraulics' order placement and fulfilment became, in effect, one continuous flow inside NetSuite, that was traditional automation: the integration lives in the platform's own architecture, robust and invisible, and it does not care what the screen looks like. It is the sturdier approach — but it requires engineering to build and to change, which is exactly why companies so often reach for a bot instead.

RPA works at the surface. It does not go through the plumbing; it sits on top, driving the same screens a person would. That is why it is fast to deploy and why it is brittle: it trades the robustness of a real integration for the speed of skipping one. Everything RPA touches must be structured and rule-bound — a fixed sequence of steps with no ambiguity — because the bot is following a script, not exercising judgment.

AI works on judgment. Where RPA needs a clear rule for every step, AI is the tool for the steps that have no fixed rule — reading an unstructured email and working out what the customer actually wants, classifying a document, resolving an ambiguous case where two reasonable people might decide differently. This is the boundary this whole module is organized around, and it is the same boundary you drew by hand in Lab 1 between the stable-rules decision point and the step that needs judgment. This week teaches only the rules-based side. The judgment side — where cognitive technologies take over — is the shift we preview but do not teach here; its full treatment is Week 7, and you build an agent for it in Lab 3. For now, simply hold the line: RPA is for what has a rule; AI is for what does not.

Make the three concrete on one familiar step, order entry in an order-to-cash flow. The traditional-automation version is what Bailey has: the storefront and the finance system are one platform, so a new order becomes an inventory reservation and a ledger entry underneath the interface, with no screen involved. The RPA version is what a company without that integration does: a bot logs into the order tool, reads the order fields, and types them into the separate finance system, mimicking the clerk — same outcome, but riding on the surface and liable to break when either screen changes. The AI version is not order entry at all; it is the step next to it — the order that arrives as a free-text email, or the dispute that does not fit any rule — where something has to interpret rather than transcribe. Same flow, three different tools, chosen by what each step actually requires.

Approach	Where it acts	What it handles	Anchored in this week
Traditional / back-end automation	In the plumbing — APIs, database, native integration	Structured data through proper system interfaces; robust, but needs engineering to change	Bailey Hydraulics' native NetSuite Order-to-Cash flow
RPA	At the surface — the same screens a person uses	Repetitive, rule-based steps with no judgement; fast to build, brittle when a screen changes	Thermo Fisher's order-entry bot moving order data across systems
AI	On judgement — unstructured, ambiguous input	Cases with no fixed rule: reading intent, classifying, resolving ambiguity	The exception routed out of Thermo Fisher's flow (previewed; taught W7)

RPA is the middle option: rules at the surface. Traditional automation goes through the plumbing and is robust but needs engineering; AI handles judgement and unstructured input; RPA sits between them, driving screens by a fixed rule. The line between rule and judgement is the one you drew in Lab 1 — and it organizes the entire module.

First Decide the Step Should Exist: The Process Mindset

Before you automate a step, you owe it one prior question — the same one Week 2 drilled into you: should this step exist in this shape at all? RPA makes this question more urgent, not less, precisely because it is so easy. A deep integration is expensive enough that someone usually stops to ask whether the process deserves it. A bot is cheap enough to point at almost anything — including a process that should have been redesigned out of existence. The easier the tool, the sharper the discipline has to be.

This is the trap the automation literature calls the task mindset, and Tella's account of it (Chapter 2, in your optional readings) is worth carrying because it names the failure precisely. The task mindset sees a person doing a tedious job — copying data from a signed contract into the ERP, say — and reaches for a bot to do that one task faster. The CFO is happy; a few hours a week come back. But zoom out to the whole process the task lives inside, and the win is small and possibly misdirected: you have made one step of a tangled order-to-cash flow faster while leaving the tangle intact. The process mindset asks the bigger question first — what is this whole flow for, and what outcome actually matters — and only then decides which steps to automate, which to redesign, and which to route to a human. In Tella's compression of the idea, “an operationalized checklist is an automated process”: a process worth automating is one you could already write down as a clean, stable checklist, and if you cannot, that is the tell that it needs redesign before it needs a robot.

You already have the course's word for the failure. Automate a broken process and you have paved the cow path — poured concrete over a route that wandered, making it a permanent, efficient way to go nowhere sensible. Week 2 taught the cure as a sequence: obliterate, then automate. This week is the “then automate” half, and the sequence is load-bearing. RPA is what you apply after the redesign, to the stable, rules-based residue that is genuinely worth keeping — not the tool you reach for instead of redesigning. Recall Ford from Week 2: the win came from eliminating the invoice-mismatch work, not from automating the chasing of mismatches faster. RPA is for the work that survives that kind of scrutiny — the repetitive step you have decided, deliberately, should exist.

And the same instinct you learned as business first, data last applies here as a rule of order: process first, bot last. You read the process, decide what should exist, and only then point automation at what remains. A bot chosen before that diagnosis is a solution hunting for a problem — the exact consulting failure this course keeps warning against.

Automate the step only after you have decided it should exist. The task mindset bolts a bot onto a tedious step; the process mindset asks what the whole flow is for and redesigns first. Automating a broken process just paves the cow path. RPA is the “then automate” half of obliterate-then-automate — applied to the stable, rules-based residue that survives redesign, never a substitute for it.

Three Tests for a Good Candidate

So which steps survive? A high-value RPA candidate passes three tests, and you should be able to run them from memory on any step of a process map. The step must be repetitive, rules-based, and stable. Repetitive means high volume — the same work done again and again, so that a bot's fixed cost is spread across enough transactions to pay for itself. Rules-based means the logic is clear and complete — every case can be decided by an explicit rule, with no point where a human would have to exercise judgment. Stable means the process and the screens it runs on do not change often — because every change is a chance to break the brittle surface-level script the bot depends on.

The RPA candidate filter: three tests

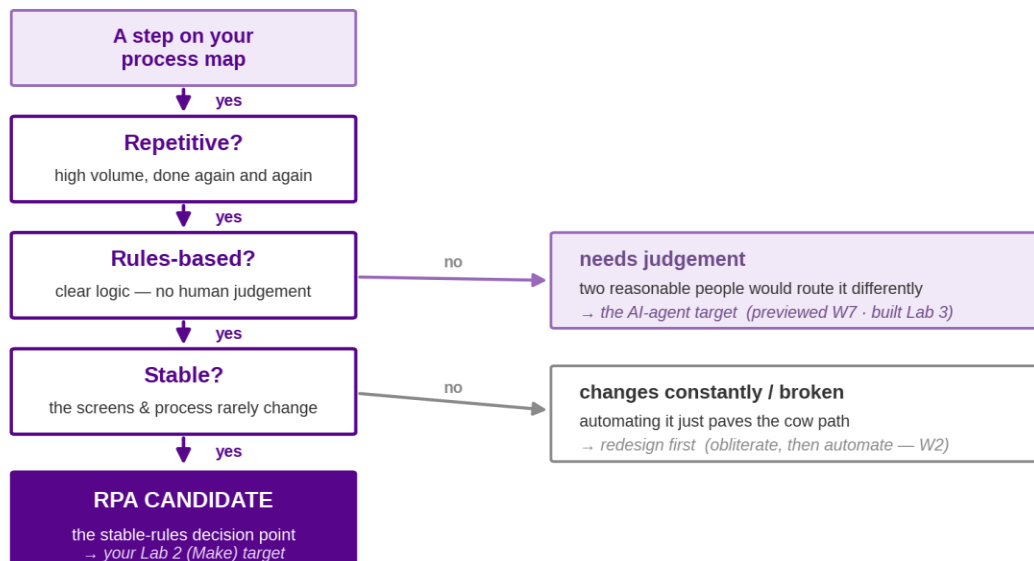


Figure 3. The RPA candidate filter: a step must pass all three tests. The candidate is the stable-rules decision point from Lab 1.

Read the filter in Figure 3 against your own Lab 1 map, because this is where the whole capstone connects. In Lab 1 you marked three diagnostic points on your process, and two of them sat on decision points: one with stable rules, one that needs judgment. The stable-rules decision point is exactly what this filter selects — it is your RPA candidate, and it is the target you will automate in Lab 2 with Make. The judgment decision point fails the second test and

branches off to become your AI-agent target, which you preview now and build in Lab 3. And a step that fails the third test — one that changes constantly, or is simply broken — is not an RPA candidate at all; it is a redesign job first. This week does not ask you to build anything; it asks you to run this filter across your mapped process and produce the RPA-candidate scan that becomes Board Memo §4. You are formalizing the decision-point call you already made by hand.

Test	What qualifies	What disqualifies	Anchored exemplar
Repetitive	High, recurring volume — the same work many times	One-off or rare work, where a bot never pays back	Thermo Fisher: order entry across millions of transactions a year
Rules-based	Every case decided by explicit, complete logic	Any point that needs human judgement or interpretation	An ambiguous customer case — routed out to a human, not the bot
Stable	The process and its screens rarely change	Frequently changing screens or an unsettled process	A broken flow — redesign first (obliterate, then automate, W2)

Notice what the three tests do together: they route a process map into three destinations. Repetitive-and-rules-based-and-stable goes to RPA. Rules-based-but-needs-judgment goes to AI. Not-yet-stable goes back to redesign. That routing is the practical core of the module, and it is why the RPA-candidate scan is diagnostic rather than a build — you are sorting the work before anyone automates a line of it.

Run the tests once, slowly, on a step you will recognize from Lab 1: the credit check on an incoming order. Is it repetitive? Yes — every order passes through it, so the volume is there. Is it rules-based? That depends entirely on how the check is defined, and this is where the judgment lives. If the rule is “approve when the customer is inside their credit limit and has no overdue invoices, otherwise hold for review,” then the approve/hold routing is a complete, explicit rule and the step passes — it is a stable-rules decision point and a clean RPA candidate. But if the real check is “approve unless the account manager has a bad feeling about this customer's cash position,” there is no rule at all, only judgment, and the same step fails the second test and belongs to AI or a human, not a bot. Is it stable? If the credit policy and the screens it runs on hold steady, yes; if finance rewrites the policy every quarter, the bot will need constant rework and the honest answer is not yet. The lesson is that “credit check” is not automatically an RPA candidate — the exact same-named step can pass or fail depending on how much judgment is actually hiding inside it. Your job in the scan is to look past the label and test the real logic.

Repetitive, rules-based, stable — all three, or it is not a candidate. Volume makes the bot pay; a complete rule makes it safe; stability keeps it from breaking. The step that passes all three is the stable-rules decision point you found in Lab 1 — your Lab 2 target. The judgment step branches to AI (W7, Lab 3); the unstable one goes back to redesign.

Attended and Unattended — and Where the Human Stays in the Loop

Once you have a candidate, there is a design choice about how the bot runs, and it matters because it decides where a person still sits. An attended bot runs alongside a human, triggered by them and working on their machine — a customer-care agent presses a key and the bot fills nine screens while the agent watches and stays ready to take over. An unattended bot runs on its own, on a schedule or off a queue, with no human present — overnight it clears a backlog of routine orders while everyone is asleep. Attended automation keeps a person in the loop by design and suits work where a human judgment or a human hand-off is never far away; unattended automation removes the person from the routine entirely and suits high-volume, back-office work that genuinely needs no supervision case by case. The Thermo Fisher pattern you are about to read blends both: unattended order entry running at scale, with the exceptions attended by a human in case management.

This is the first appearance of a question that will grow louder every week of this course: even for a rules-based bot, where should a human stay in the loop on purpose? It is tempting to read “rules-based” as “no human needed,” but that is a trap. A bot follows its rule with perfect confidence even when the rule is wrong for the case in front of it — it does not pause, does not doubt, does not notice the thing a person would have caught out of the corner of their eye. So the mature move is often to automate the routine and deliberately route the edge cases, the high-value transactions, or the irreversible actions to a human checkpoint. You will feel this directly in Lab 2, where the deliverable asks you not just to build an automation but to break it and then state the human oversight you would build in. And it is a governance question as much as a design one — Week 5 asks explicitly when a human should be intentionally integrated into an automated workflow even when the rules would let the bot run alone. For now, plant the flag: choosing where the human stays is part of choosing the candidate, not an afterthought.

Decide how the bot runs — and where a human stays — as part of choosing the candidate. Attended bots run beside a person and keep a hand on the wheel; unattended bots run alone on high-volume routine. “Rules-based” does not mean “no human needed” — a bot follows a wrong rule with perfect confidence, so route edge cases, high-value transactions, and irreversible actions to a human checkpoint. Lab 2 makes you name that oversight; Week 5 governs it.

What a Bot Does to the Cost of a Task — and to the Person Who Did It

The reason a good candidate is worth automating is a change in cost structure, and it is worth stating precisely because it is what a board actually funds. A task done by people has a variable cost: each additional transaction consumes a little more labour, so cost rises roughly in step with volume. A task done by a bot converts most of that into a fixed cost — you pay to build and license the bot, and each additional transaction after that costs almost nothing. The marginal cost of the automated task collapses toward zero. That is why the business case sharpens as volume rises, and why the repetitive test comes first: a bot on a low-volume task is a fixed cost with nothing to spread it over.

Make the arithmetic concrete, because Lab 2 asks you for exactly this kind of simple estimate and your board memo lives or dies on it. Suppose a routine step takes a clerk three minutes and runs five hundred times a day. That is twenty-five hours of work a day — call it three to four full-time people once you count the working day. A bot that handles the rules-based bulk of it does not eliminate every minute (the exceptions still go to a human, as they should), but it can take out the great majority, and after the one-time build and the annual licence, each of those transactions costs almost nothing to run. The board case is then a straightforward comparison: the recurring labour cost removed against the one-time build plus the running licence, expressed as a payback period — which, Deloitte's 2022 respondents found, now runs on the order of twenty-two months rather than the sixteen it once did. You do not need a sophisticated model; you need an honest one, with the exception volume left in and the human oversight costed rather than wished away.

But the same sentence that describes the saving describes a person. “The marginal cost collapses toward zero” means the hours a human used to spend are no longer needed for that task — and the honest reading of this week is that RPA changes, and sometimes eliminates, back-office roles. The comfortable narrative is that those people are “redeployed to higher-value work.” Sometimes that is real: freed from moving data between screens, a customer-care specialist can spend the time on the judgment-heavy cases a bot cannot touch, which is genuinely better work. Sometimes it is a convenient story a firm tells to avoid a harder conversation. You should hold the question open rather than resolve it with a slogan: when RPA removes the routine part of a role, what does the firm owe the person who held it, and is the redeployment a commitment the firm actually keeps or a phrase in a press release? Deloitte's survey (2022) found that most organizations scaling automation had not even calculated how much of their workforce their automation would affect — which tells you how often the question goes unasked. Carry it; Week 13 returns to the human side of automation in full, and your Board Memo §7 will have to answer it for your own company rather than wave at it.

A bot converts a variable labour cost into a near-zero marginal cost — and that is a person's hours. The business case sharpens with volume because each automated transaction costs almost nothing after build. The same fact means routine roles change or disappear. “Redeployment to higher-value work” is sometimes real and sometimes a convenient narrative; hold the obligation to the affected person open rather than closing it with a slogan.

Where the Repetitive Work Comes From: MRP and S&OP

It helps to know where all this repetitive, rules-based, exception-laden work is generated in the first place, because it is not random — it is manufactured by the planning layer that sits on top of the ERP data model. Two long-established operations-management systems run that layer. Materials Requirements Planning, or MRP — the discipline set out by Joseph Orlicky in 1975 as the origin reference — takes a production plan and explodes it into the timed orders for every component and material needed to build it. Sales and Operations Planning, or S&OP — associated in its origins with George Ling and Walter Goddard's 1988 account — is the higher, cross-functional cadence that reconciles the demand plan with the supply plan so the company

commits to one set of numbers. Together they are how a manufacturer decides what to make and buy, and when.

You have already met their signature rhythm under its true name. This is the batch-push paradigm from Weeks 1 and 2: plans are computed overnight, cascade downstream, and humans spend the morning firefighting the exceptions. MRP and S&OP are the engines that run that batch. And the morning firefighting is the point — the overnight run throws off a steady stream of exceptions (a purchase order that needs releasing, a mismatch that needs reconciling, a status that needs updating across two systems), and that exception work is exactly the repetitive, rules-based residue RPA first found traction on.

Picture one concrete exception, because it is the archetype of an RPA candidate. The overnight MRP run notices a component has dropped below its reorder point and generates a planned order; in the morning a buyer opens it, checks it against the approved supplier and the standing rule, and releases it as a purchase order, copying a few fields between the planning screen and the purchasing screen as they go. Multiply that by hundreds of components a day and you have precisely a repetitive, rules-based, stable step — high volume, a clear rule for the routine cases, screens that rarely change — with the genuinely unusual cases (a supplier on hold, a suspicious quantity) peeled off for a human. That is the RPA filter landing on the exact work the planning layer manufactures every night.

So keep the causal chain straight: RPA does not fix the batch-push rhythm; it automates the clerical firefighting the rhythm produces. That is precisely why it is tactical — it makes the symptoms cheaper without changing the paradigm underneath. The deeper cure — shifting toward pull-based, event-driven operations where decisions happen at the point of variance rather than the morning after — is the redesign Week 2 pointed at and the direction Modules 3 and 4 build toward. Hold that as a preview, not this week's lesson.

MRP and S&OP are the batch-push engines that generate the exception work RPA targets. MRP (Orlicky, 1975) explodes a plan into component orders; S&OP (Ling and Goddard, 1988) reconciles demand with supply. Their overnight, cascade-then-firefight rhythm throws off the repetitive, rules-based exceptions a bot handles well. RPA automates that firefighting; it does not change the batch-push paradigm — that is the later, deeper work.

Automate the Rule, Route the Exception: Thermo Fisher

Now walk the primary case, because it shows the whole logic of the week in one operational picture — and it is deliberately ordinary. Thermo Fisher Scientific is one of the world's largest suppliers of scientific instruments and reagents, assembled over years of mergers and running, as a result, on a sprawl of disparate ERPs, CRMs, and middleware. Across order entry, sales, and customer service it handles more than ten million transactions and interactions a year (Olavsrud, CIO, 2022). This is not a lighthouse AI story; it is the kind of high-volume, back-office reality that most real automation lands in, which is exactly why it is the primary case.

The problem, in the language you now have. Because the systems were never integrated, the human was the integration. A customer-care representative answering a simple question — where is my order, is this product available — sometimes navigated nine different systems and nine different screens, and a routine call could run past ten minutes. That is the Week 1 symptom in its purest form: a significant part of the job was carrying data between systems that could not talk to each other. Read through the Four Lenses, it is Frustration (staff and customers both) and Time (ten minutes for a one-line answer), and it sits first on the Operations thread with an immediate Customer face.

What happened. Thermo Fisher's Project Northstar attacked this without the months-long ERP consolidation the old instinct would have demanded — the instinct one of its own IT leaders described as the reflex of a few years earlier, to put everyone on a single ERP and spend years and millions consolidating. Instead it built a set of discrete modules on top of the existing sprawl: an order-entry automation that combines RPA, machine learning, and OCR to ingest incoming orders — including faxed and emailed ones — and move the data across the systems of record automatically; a case-management module that auto-assigns customer emails to the right team; a single search interface that pulls order status, availability, pricing, and invoices from one place instead of nine; and a customer-insights layer over a central data lake. The bot does the cross-screen carrying the humans had been doing, and — crucially — the flow routes the order exceptions out to case management rather than forcing the bot to handle them.

Notice the shape, because it is the whole week in one diagram: the repetitive, rules-based order entry is automated, and the ambiguous exception is routed to where judgment lives. That is the RPA-candidate filter enacted in production, exceptions and all — and note that the order-entry module was still being iterated even after launch, which is the honest texture of real automation rather than a finished miracle. It is also worth how the work was scoped: Thermo Fisher mapped its processes down to the click level with a delivery partner before automating anything, which is process-first, bot-last in practice, not slogan. You cannot write a bot's fixed script until you know, click by click, exactly what the stable rule is — mapping is how you find it, which is why Lab 1 came before this week.

The lesson. The value showed up on the two threads Module 2 promised it would. After the customer-engagement work was fully deployed, average call hold times fell by about 27 percent (Olavsrud, CIO, 2022) — a direct Customer gain — and processing orders faster accelerated the company's cash flow, a direct Cash gain. Neither came from a bot doing something clever; both came from a bot doing something dull, at scale, on a step that was repetitive, rules-based, and stable enough to automate — while the step that was none of those was deliberately handed to a human. That division of labour is the entire craft of the week. Read the full account here:

<https://www.cio.com/article/404535/thermo-fisher-transforms-its-customer-experience.html>

Thermo Fisher automated the rule and routed the exception. Reps had been the integration between nine screens; an RPA-and-OCR order-entry flow took over the repetitive cross-system carrying and routed ambiguous exceptions to a human. Call hold times fell about 27% (Customer) and order processing accelerated cash flow (Cash). The win came from a dull step done at scale — the RPA filter in production.

What “Working” Looks Like First: Bailey Hydraulics

Set beside Thermo Fisher a quieter case that answers a different question: what does a process look like when the backbone already works — before you ask what a bot would add? Bailey Hydraulics is the second course archetype made real: a mid-market manufacturer of mobile hydraulic components, running its business on NetSuite. It is the baseline, not the transformation, and that is the point of including it.

The problem it started from is one you can now name instantly. Before NetSuite, Bailey ran on a heavily customized legacy ERP with its web storefront bolted on separately, so orders had to be moved by hand between the website and the back-end system — the human as integration again, and the financial close dragging to twenty days. What happened is not RPA at all: consolidating onto one platform gave the order-to-cash flow a single source of truth, so order placement and fulfilment became effectively continuous inside the system, and the financial close fell from twenty days to three (Oracle NetSuite). Staff could pull their own reports in minutes instead of waiting weeks on an IT ticket.

The lesson is the one this case exists to teach, and it is a caution about your own RPA-candidate scan. Bailey's gains came from traditional, native integration inside the platform — the plumbing — not from a surface-level bot. When the backbone genuinely works, a whole class of “move this data between two systems” tasks simply disappears, because the systems are now one. So before you scan a working process for RPA candidates, ask what the native integration already handles: pointing a bot at a hand-off that a real integration should own is the surface-level patch when the structural fix is available. Bailey is the working-backbone baseline against which you ask the sharper question — given that the backbone works, what specifically would RPA still change here? Often the honest answer is: less than you would think, and only at the residual manual edges. Read the baseline here:

<https://www.netsuite.com/portal/resource/articles/ecommerce/bailey-hydraulics-pumps-up-ecommerce-business-with-netsuite.shtml>

A working backbone removes work before any bot is considered. Bailey's 20-day-to-3-day close and end-to-end order flow came from native NetSuite integration — the plumbing — not RPA. When systems become one, whole “carry data between screens” tasks vanish. Scan a working process by asking what integration already owns before pointing a bot at a seam that a real integration should.

What the Market Is Actually Doing

Zoom out from two companies to the field, because your board will want to know that this is settled practice, not a bet. Deloitte's 2022 Global Intelligent Automation survey, which reached 479 executives across 35 countries, is the reference for the state of play — and every figure here is Deloitte's own, not blended with any other firm's numbers. The headline is that RPA has crossed into the mainstream: about 74 percent of respondents reported already implementing RPA (Deloitte, 2022). It is no longer an experiment; it is infrastructure.

The business case, in Deloitte's numbers, is real but has matured into something more sober than the early hype. Organizations reported expecting average cost reductions around 31 percent over three years, and those past the piloting stage reported achieving about 32 percent (Deloitte, 2022) — with one financial-services executive describing a cost reduction of over 70 percent in a targeted area, but only after radically re-engineering the process and combining several tools, which is this week's obliterate-then-automate lesson stated as a result. The cost of that ambition also rose: the average estimated payback period lengthened from about 16 months to about 22 months (Deloitte, 2022), as organizations invested more to get more. Automation got more valuable and more expensive at the same time.

Two of Deloitte's findings are worth carrying specifically because they connect backward and forward. First, the barriers to scaling: the top ones reported were process fragmentation, lack of a clear vision, lack of IT readiness, and resistance to change (Deloitte, 2022) — notice that none of them is the technology. This is the Module 1 refrain again (a backbone problem is a process and organization problem, not a tools problem), and it previews Week 5's answer, the Center of Excellence. Second, the trajectory: a large majority of more mature adopters were already moving from task-level bots toward end-to-end automation — roughly 44 percent already doing it and another 48 percent planning to within three years (Deloitte, 2022). That is the arc of this course in the survey data: RPA is where you start, not where you stop. One more finding is a quiet rebuke worth heeding — over half of scaling organizations had not calculated the share of their workforce automation would affect, and many had not measured their own cost savings at all (Deloitte, 2022). If your board memo quantifies its prize, as Lab 1 taught you to, you are already ahead of most of the market.

RPA is mainstream, its payback is real but slower, and its hardest barriers are not technical. About 74% of Deloitte's 2022 respondents were already running RPA; cost reductions clustered near a third, with payback stretching from ~16 to ~22 months as ambition grew. The top scaling barriers were process fragmentation, vision, IT readiness, and resistance — an organization problem, previewing Week 5's Center of Excellence. Most mature adopters were already moving from task bots toward end-to-end automation.

Where This Points: The Threads, the Labs, and the Capstone

Pull the week together on the three axes this course always uses. On the threads, RPA's home is Operations and Supply Chain — the repetitive, rules-based operational work — but you have watched it land on Cash (Thermo Fisher's accelerated cash flow, Bailey's faster close) and on

Customer (Thermo Fisher's shorter hold times). When you scan your own company, tag each candidate with the thread it moves, because a board funds a thread outcome, not a bot.

On the labs, this is the hinge week. The stable-rules decision point you marked in Lab 1 is this week's RPA candidate, and it becomes the exact step you automate in Lab 2 with Make — build it, then break it, so the brittleness this note described stops being abstract. The judgment decision point you also marked is not for RPA; it is handed forward to become your AI-agent target — you pin it down in Week 7 and build it in Lab 3 (Week 9). Keep both in view, but only the first is this week's business.

On the capstone, your deliverable is a short RPA-candidate scan for your company: which repetitive, rules-based, stable steps qualify, and why — run the Figure 3 filter across your Lab 1 map and write it up. That scan is Board Memo §4 (Automation Journey), and on the final board deck it fills the Automation opportunity slide (S6) — the case for where automation adds value, still inside the Diagnose phase (Milestone 1) of the project. You are not yet proposing how to run automation across the enterprise; you are naming, with evidence, where it belongs.

Two things are deliberately left for later, and you should hand them forward cleanly rather than reaching for them now. Why RPA programmes stall once you have more than a handful of bots — the brittleness at scale — and the Center of Excellence that governs the jump to enterprise scale are next week, Week 5; that is also where the fourth study question below points. The move from rules to genuine machine judgment — intelligent automation — is Week 7, where you also pin down the one step that needs AI judgment; the agent's architecture, the SPAR framework, follows in Week 8, and you build and break the agent in Lab 3, Week 9. This week teaches only the first rung: the rule-based robot, and the discipline of choosing where it belongs.

This week names where a bot belongs; the next steps scale it and make it intelligent.

Thread-tag every candidate you find, because boards fund outcomes, not bots. Your Lab 1 stable-rules decision point is the RPA candidate and the Lab 2 build; the judgment point is handed to AI (W7, built in Lab 3, Week 9). The scan is Board Memo §4 and the Automation-opportunity slide (S6). Scaling and governance are Week 5; intelligent automation is Week 7.

Writing Your RPA-Candidate Scan

Turn all of this into the deliverable, because a scan is only useful if it is written down in a form a board could act on. Take the friction chain and process map you built in Lab 1 and walk it step by step, running the three tests on each candidate step and recording, for the ones that pass, four things: the step, why it passes (repetitive, rules-based, stable), the thread it moves, and a rough sense of the prize. Keep it short and honest — two or three real candidates, not a wish list — and be explicit about the steps you rejected and why, because a scan that only says yes has not done the diagnostic work.

Here is the shape, using the inventory-to-cash chain from the Lab 1 skeletons as the worked example. Suppose your map shows: inaccurate stock leads to overselling, an order is rejected at fulfilment, the rejection is reworked by hand, invoicing is delayed, and cash arrives late. Walk

it. The stock-accuracy problem is not an RPA candidate — it is a data and process problem, a redesign job, and pointing a bot at it would pave the cow path. The fulfilment credit-or-stock check, if it runs on a clear rule, is a stable-rules decision point: repetitive, rules-based, stable — an RPA candidate, on the Operations thread, whose prize is fewer wrongly rejected orders and faster invoicing. The manual rework of a rejected order — someone re-keying it across two systems after the hold clears — is a strong candidate too: high volume, a fixed sequence of keystrokes, stable screens; its thread is Cash, because compressing that rework is days of cash conversion recovered. And the genuinely ambiguous rejection, the one where a person has to decide whether to make an exception for a good customer, is not an RPA candidate at all — it is your judgment step, flagged for the AI work in Lab 3.

Written up, that is three lines a board can read: two RPA candidates with their threads and prizes, and one honest “not this — redesign first” and one “not this — needs judgment.” That is your Board Memo §4, and it is worth more than a longer list precisely because it shows you sorted the work rather than automating everything in sight. The scan is the argument; the build is Lab 2.

The scan names two or three real candidates — and says what you rejected and why.

Walk your Lab 1 chain step by step, run the three tests, and record for each passing step the step, why it passes, its thread, and the prize. Rejecting the stock problem (redesign first) and the ambiguous rejection (judgment) is as much the work as naming the candidates. Short and honest beats long and indiscriminate — this is Board Memo §4.

Study Questions

Bring reasoned answers; they are designed to be argued with cases and frameworks, not recited.

1. List the three criteria for identifying a high-value RPA candidate process, and use one step from your own Lab 1 map to show a step that passes all three — and one that fails.
2. Using the Bailey Hydraulics case as the baseline, describe what Order-to-Cash looks like when the integration backbone is working. What, specifically, would RPA change in that picture — and what has native integration already made unnecessary?
3. How does RPA change the cost structure of a repetitive task, and what does that imply for the workforce? Take a position on whether “redeployment to higher-value work” is a genuine commitment or a convenient narrative, and say what would make the difference.
4. What is the single biggest governance risk of deploying RPA without a Center of Excellence? Answer from what you know now, then note what you expect Week 5 to add.

Running Glossary

New this week, plus the carried terms this note builds on.

Robotic Process Automation (RPA). Software that automates a repetitive, rules-based task by operating a company's existing applications through their user interface — driving the same screens a person would — rather than through a deep integration underneath. Fast to deploy and able to bridge un-integrated systems, but brittle when a screen changes.

Traditional / back-end automation. Automation built through systems' proper interfaces — an API, a database write, a native platform integration — so a transaction flows beneath the user interface. More robust than RPA and independent of the screen, but it requires engineering to build and to change.

RPA candidate criteria. The three tests a step must pass to be worth automating with a bot: repetitive (high, recurring volume), rules-based (every case decided by explicit logic, no judgement), and stable (the process and screens rarely change).

MRP (Materials Requirements Planning). The planning method that explodes a production plan into the timed orders for every component and material required to build it (origin reference: Orlicky, 1975). Part of the batch-push planning layer on top of the ERP.

S&OP (Sales and Operations Planning). The cross-functional planning cadence that reconciles the demand plan with the supply plan so the company commits to one set of numbers (origin reference: Ling and Goddard, 1988).

Working-backbone baseline. A process running on an integration backbone that already functions — used as the reference point for asking what automation would still add, rather than assuming a bot is the first move.

Batch-push planning. The paradigm in which plans are computed overnight, cascade downstream, and humans firefight the exceptions in the morning — the rhythm MRP and S&OP run, and the source of the repetitive exception work RPA targets. (Introduced Weeks 1–2; its cure, pull-based event-driven operations, is later work.)

Control point. A moment where the business commits and a new authoritative fact is created — a PO issued, an invoice posted, credit approved. Where governance and automation attach. (Introduced Week 1.)

Decision point. A moment where the routing choice is made — approve or reject, route one way or another; a named BPMN gateway. Two of Lab 1's three diagnostic points sit here; the stable-rules one is this week's RPA candidate. (Introduced Week 1.)

Leverage gap. The distance between a backbone that works technically and a backbone that delivers business value. Closing it, layer by layer, is the work of the course. (Introduced Week 1.)